



Good

Evening

Everyone ☺

FIND SUM OF ALL SUBARRAYS SUM

Given an array of integers, find the total sum of all possible subarrays.

Input [3 2 5]

$$[3] \quad \Rightarrow \quad 3 + 10 + 5 + 2 + 7 + 5 \quad \Rightarrow \quad \underline{29}$$

[3 2 5]

[3 2]

[2]

[2 5]

[5]

Brute force - generate all subarrays and calculate sum.

Code:

function SubArraySumBrute (arr[], n) {

TC : $O(n^3)$
SC : $O(1)$

total = 0

for (i → 0 to n-1) {

for (j → i to n-1) {

subarray_sum = 0

for (k → i to j) {

subarray_sum += arr[k]

}

total = total + subarray_sum

}

}

print (total)

}

SUBARRAYS SUM - PREFIX SUM

1) Calculate Prefix Sum array for original array

$$Pf[i] \Rightarrow Pf[i-1] + A[i]$$

Code.

```
function totalSubArrayPrefixSum( arr[], n) {
```

```
    prefix[n];
```

TC : $O(n^2)$
SC : $O(n)$

```
    prefix[0] = arr[0]
```

```
    for (i = 1 to n-1)
```

```
        prefix[i] = prefix[i-1] + arr[i]
```

```
    }
```

```
    total = 0
```

```
    for (i = 0 to n-1) {
```

```
        for (j = i to n-1) {
```

```
            sum = 0
```

```
            start = i
```

end = j

if (start == 0) {

sum = prefix[end]

}

else

sum = prefix[end] - prefix[start-1]

}

total = total + sum

}

}

print (total)

}

SUBARRAYS SUM - Carry Forward

$A[] =$

0	1	2	3
-4	1	3	2

$i = 0$

$j = 0$

$k \% \text{ sum} \Rightarrow A[0]$

$j = 1$

$k \% \text{ sum} \Rightarrow A[0] + A[1] - 2$

$j = 2$

$k \% \text{ sum} \Rightarrow A[0] + A[1] + A[2]$

$j = 3$

$k \% \text{ sum} \Rightarrow A[0] + A[1] + A[2] + A[3]$

$A[] \Rightarrow$

-4	1	3	2
----	---	---	---

$i = 0$

currSum = 0

j	Subarray index	currSum	value	total
0	[0 0]	$+ = A[0]$	-4	-4
1	[0 1]	$+ = A[1]$	$-4 + 1 = -3$	-7
2	[0 2]	$+ = A[2]$	$-4 + 1 + 3 = 0$	-7
3	[0 3]	$+ = A[3]$	$0 + 2 = 2$	-5

$i=1$

curSum = 0

j	Subarray	curSum Value	Total
1	[1 1]	$t = A[1] \Rightarrow 1$	-4
2	[1 2]	$t = A[2] \Rightarrow 1+3=4$	0
3	[1 3]	$t = A[3] \Rightarrow 1+3+2=6$	6

$i=2$

curSum = 0

j	Subarray index	curSum Value	Total
2	[2 2]	$t = A[2] = 2$	9
3	[2 3]	$t = A[3] = 3+2 \Rightarrow 5$	14

$i=3$

curSum = 0

j	Subarray	curSum	
3	[3 3]	$t = A[3] = 2$	$\Rightarrow 16$

Code

function totalSubArraySum (arr[], n) {

total = 0

TC : $O(n^2)$
SC : $O(1)$

for (i = 0 to n-1) {

cursum = 0

for (j = i to n-1) {

cursum = cursum + arr[j]

total = total + cursum

}

}

print (total)

}

$$[1 \ 2 \ 2]$$

Total = 0

$i \Rightarrow 0 \text{ to } 2$

$i=0$, $currentsum=0$
↓

$j=0$
 $[0 \ 0] \Rightarrow currentsum + A[0] \Rightarrow \underline{1}$ 1

$j=1$ $[0 \ 1] \Rightarrow 1 + A[1] \Rightarrow \underline{3}$ 4

$j=2$ $[0 \ 2] \Rightarrow 3 + A[2] \Rightarrow \underline{6}$ 10

$i=1$ $currentsum=0$

$j=1$

$[1 \ 1] \Rightarrow currentsum + A[1]$ 12
 $0 + 2 = \underline{2}$

$j=2$ $[1 \ 2] \Rightarrow currentsum + A[2]$ 17
 $2 + 3 = \textcircled{5}$

$i=2$ $currentsum=0$

$[2 \ 2] \Rightarrow 0 + 2 \Rightarrow \underline{2}$ 20 ←

SUBARRAYS SUMS - Contribution Technique

$$\text{Total} = \underline{20.}$$

[1 2 3]

[1]

[1 2]

[1 2 3]

[2]

[2 3]

[3]

$$(1 \times 3) + (2 \times 4) + (3 \times 3) \Rightarrow$$

$$3 + 8 + 9 \Rightarrow \underline{20.}$$

Quiz! In how many subarrays index 1 will be present?

A : $\begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \\ [3 & -2 & 4 & -1 & 2 & 6] \end{matrix}$

Start

end

3

-2

-2

4

-1

2

6

2×2

$= 10$

$[3 \ -2]$

$[3 \ -2 \ 4]$

$[3 \ -2 \ 4 \ -1]$

$[3 \ -2 \ 4 \ -1 \ 2]$

$[3 \ -2 \ 4 \ -1 \ 2 \ 6]$

$[-2]$

$[-2 \ 4]$

$[-2 \ 4 \ -1]$

$[-2 \ 4 \ -1 \ 2]$

$[-2 \ 4 \ -1 \ 2 \ 6]$

Quiz 2 In how many subarrays index 2 will be present?

A : $[3 \quad -2 \quad 4 \quad -1 \quad 2 \quad 6]$

0 1 2 3 4 5

Start	↓ indices	end
3	(0)	4 (2)
-2	(1)	-1 (3)
4	(2)	2 (4)
-		6 (5)

[0-2] (3) starts x 4 ends
[0-3]
[0-4]
[0-5] $\Rightarrow$ 12 subarrays.

[1-2]
[1-3]
[1-4]
[1-5] $\Rightarrow$ 12 subarrays.

[2-2]
[2-3]
[2-4]
[2-5]

Contd.

If you have array of 0 ⁱ n

You need find out the contribution of index i

Start $\Rightarrow$ [0 to i] $\Rightarrow i - 0 + 1 \Rightarrow (i+1)$

end $\Rightarrow$ [i to n-1] $\Rightarrow n - 1 - i + 1 \Rightarrow n - i$

Contribution of i = $(i+1) * (n-i)$

Contribution in sum of element at index i

$$\text{arr}[i] * (i+1) * (n-i)$$

Code:

```
function SumOf All SubArrays (arr[]) {
```

```
    n = arr.length();  
    total = 0;
```

TC: $O(n)$

SC: $O(1)$

```
    for (i → 0 to n-1) {
```

```
        total += arr[i] * (i+1) * (n-i)
```

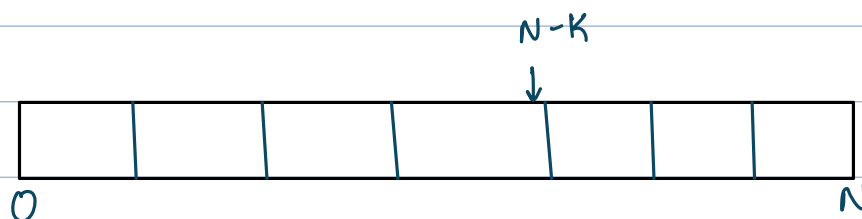
```
    }
```

```
    print (total)
```

```
}
```

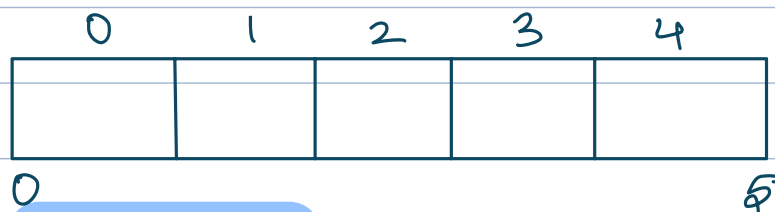
COUNT OF SUBARRAYS OF LENGTH K

Number of subarrays of length K = Total number of start indices of subarrays of length K .



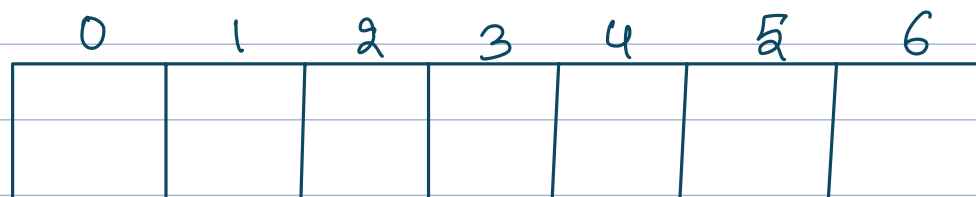
$$[0 \rightarrow N-K]$$

$$\Rightarrow \underline{N-K+1}$$



$$K=2$$

$$\underline{[0-3]}$$



$$\underline{N=7}$$

$$\underline{K=2}$$

$$7-2+1$$

$$\underline{=6}$$

Quiz 3.

Given $N=7$, $K=4$. What will be total number of subarrays of length K .

$$7 - 4 + 1 \Rightarrow \underline{4}$$

MAX SUBARRAY WITH LENGTH K

Given an array of N elements. Print maximum subarray sum for subarray with length = K .

$N=10$

$K=5$

0	1	2	3	4	5	6	7	8	9
-3	4	-2	5	3	-2	8	2	-1	4

Start

end

Sum

0

4

7

1

5

8

2

6

12

3

7

16

4

8

10

5

9

11

2.

code:

function maxSubArray of length k ($arr[]$, k)

ans = $-\infty$

TC : $O(N^2)$

for k elements

SC : $O(1)$

$i = 0$

$j = k-1$

while ($j < N$) {

 sum = 0

 for ($idx = i$ to j)

 sum = sum + $A[idx]$

 }

 if ($ans < sum$)

 ans = sum

 }

$i++$

$j++$

}

print (ans).

SLIDING WINDOW

0	1	2	3	4	5	6	7	8	9
-3	4	-2	5	3	-2	8	2	-1	4

$N=10$
 $K=6$

i start	j end	Sum	
0	5	5	16
1	6	Sum = arr[0] + arr[6]	
		$\Rightarrow 5 - (-3) + 8 \Rightarrow 16$	
2	7	$\Rightarrow 16 - \text{arr}[1] + \text{arr}[7]$	
		$\Rightarrow 16 - 4 + 2 \Rightarrow 14$	
3	8	$\Rightarrow 14 - \text{arr}[2] + \text{arr}[8]$	
		$\Rightarrow 14 - [-2] + (-1)$	
		$\Rightarrow 15$	
4	9	$\Rightarrow 15 - \text{arr}[3] + \text{arr}[9]$	
		$\Rightarrow 15 - 5 + 4$	
		$\Rightarrow 14$	

Code .

function maxSubarray of length k ($A[]$, N ; k) {

ans = $-\infty$

i = 0

j = k-1

sum = 0

for (idx $\rightarrow$ i to j) {

sum = sum + $A[idx]$

ans = max(ans, sum)

}

i++

j++

while (j < N) {

sum = sum + $A[j]$ - $A[i-1]$

if (ans < sum)

ans = sum

}

i++

j++

}

print (ans)

}

k iteration

$N-k$ iteration

SC : $O(1)$.

TC : $O(N-k+k)$
 $O(N)$.

